PROGRAM - 1

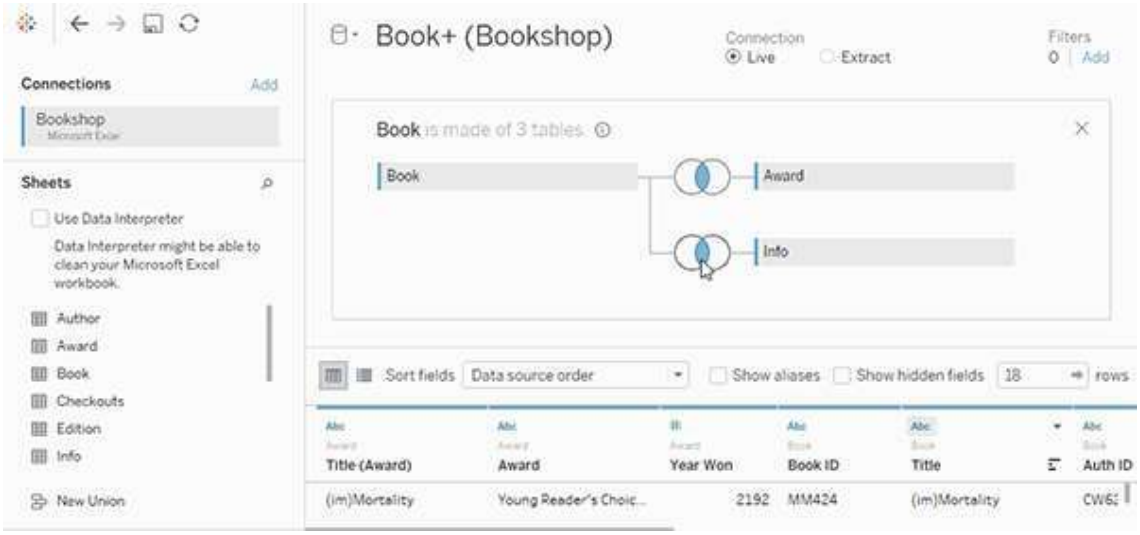
Aim:

To implement adding and joining data sources and sample datasets in Tableau.

Procedure:

1. Open Tableau and click "Connect to Data".
2. Select a sample dataset (e.g., Sample - Superstore.xls).
3. Drag the first table onto the canvas.
4. Drag the second table onto the canvas to automatically create a join.
5. Configure the join conditions.

Output:



Result:

Successfully implemented multiple data source connections and configured joins in Tableau.

PROGRAM - 2

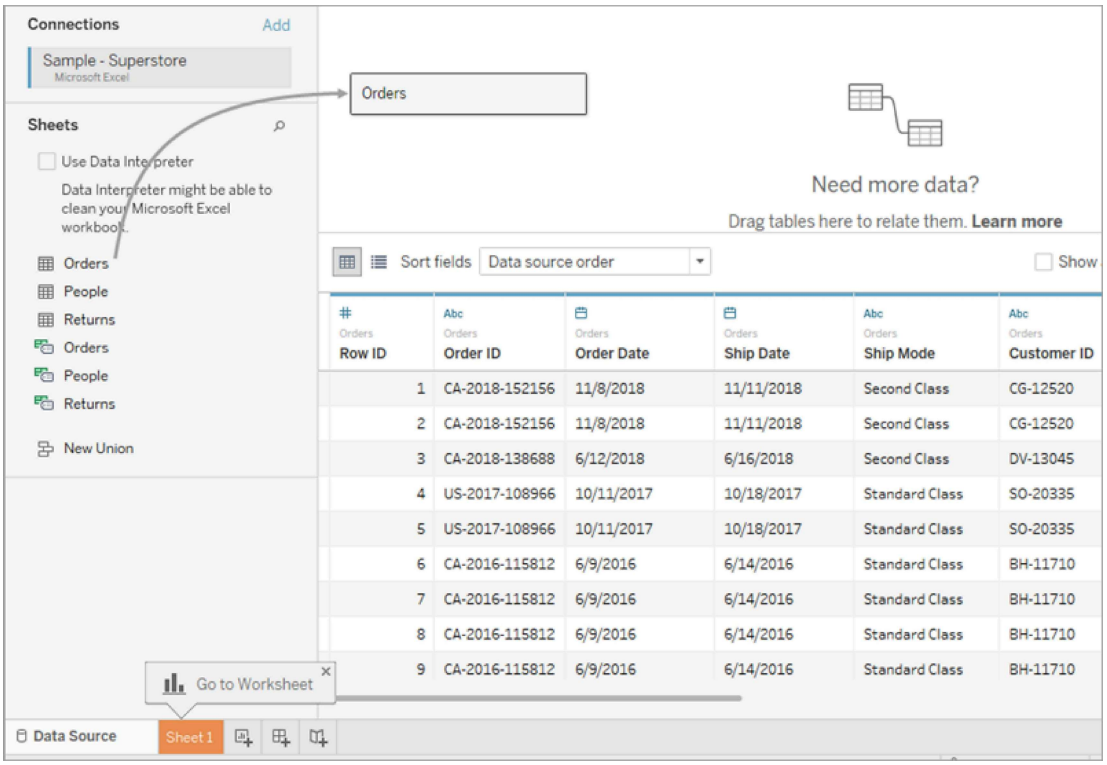
Aim:

To implement viewing, selecting, and copying data items in Tableau.

Procedure:

1. Open the Data Source tab or Worksheet.
2. Right-click the dataset in the left pane and click "View Data".
3. Select specific rows or columns.
4. Click "Copy" to copy the selected data to the clipboard.

Output:



Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID
1	CA-2018-152156	11/8/2018	11/11/2018	Second Class	CG-12520
2	CA-2018-152156	11/8/2018	11/11/2018	Second Class	CG-12520
3	CA-2018-138688	6/12/2018	6/16/2018	Second Class	DV-13045
4	US-2017-108966	10/11/2017	10/18/2017	Standard Class	SO-20335
5	US-2017-108966	10/11/2017	10/18/2017	Standard Class	SO-20335
6	CA-2016-115812	6/9/2016	6/14/2016	Standard Class	BH-11710
7	CA-2016-115812	6/9/2016	6/14/2016	Standard Class	BH-11710
8	CA-2016-115812	6/9/2016	6/14/2016	Standard Class	BH-11710
9	CA-2016-115812	6/9/2016	6/14/2016	Standard Class	BH-11710

Result:

Successfully implemented data inspection, selection, and clipboard copying actions.

PROGRAM - 3

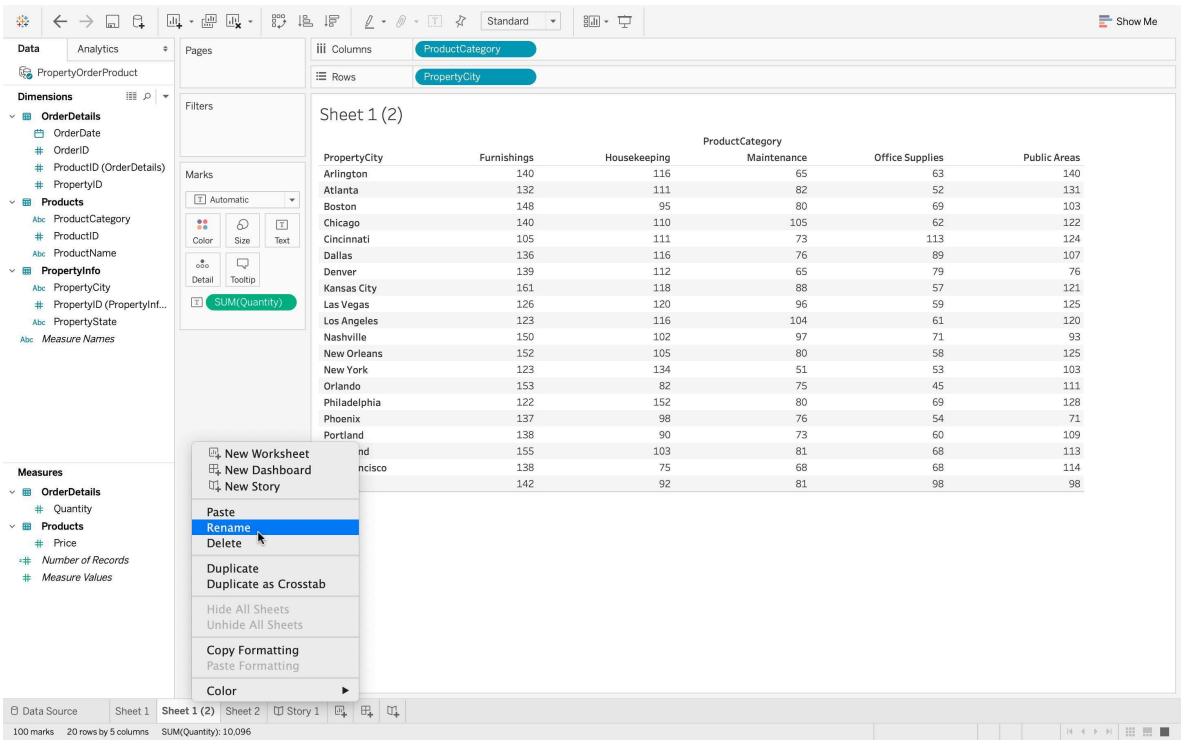
Aim:

To implement adding, duplicating, and renaming a worksheet in Tableau.

Procedure:

1. Click the "New Worksheet" icon at the bottom of the interface.
2. Right-click the newly created tab and select "Duplicate".
3. Right-click the duplicated tab and select "Rename".
4. Type the new name and press Enter.

Output:



Result:

Successfully implemented worksheet management, duplication, and renaming.

PROGRAM - 4

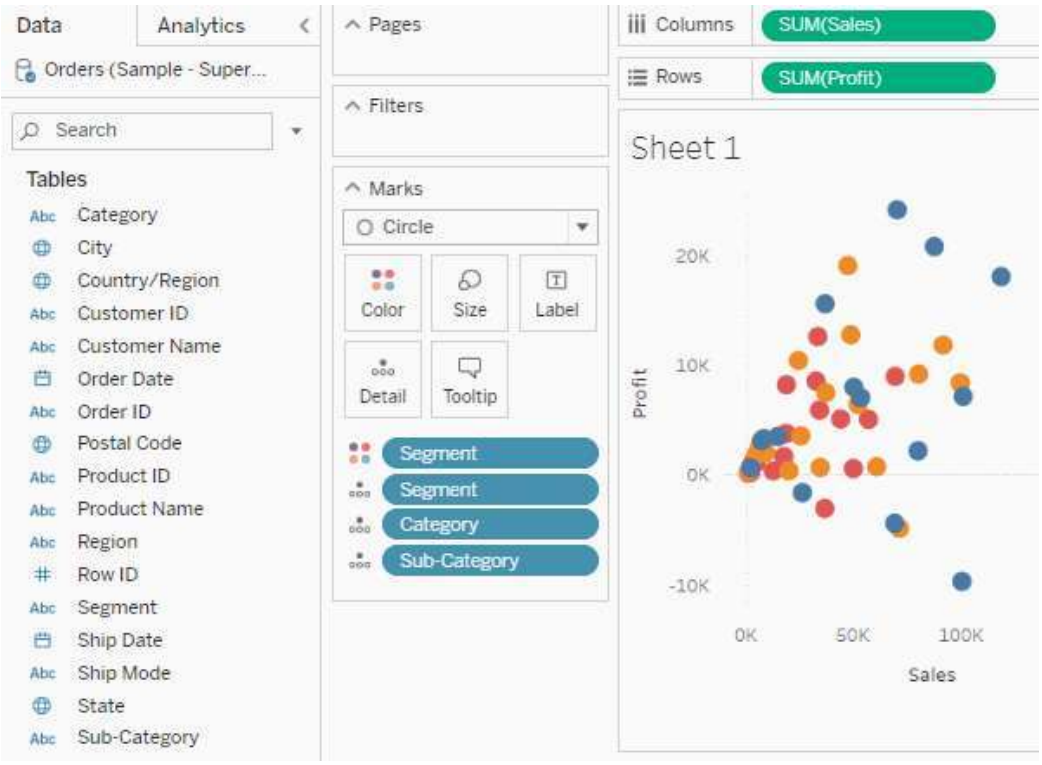
Aim:

To implement the creation and removal of hierarchies in Tableau.

Procedure:

1. In the Data Pane, drag one dimension directly on top of another.
2. Name the newly created hierarchy (e.g., Geography).
3. To remove, right-click the hierarchy and select "Remove Hierarchy".

Output:



Result:

Successfully implemented and managed logical data drill-down hierarchies.

PROGRAM - 5

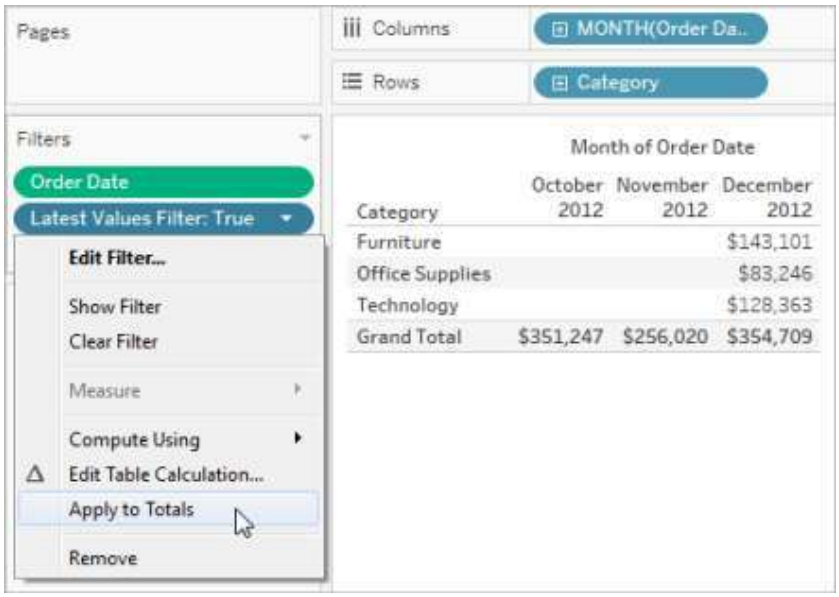
Aim:

To implement filters with conditions (Minimum Threshold and Range filters) in Tableau.

Procedure:

1. Drag a measure or date field to the Filters shelf.
2. Select the condition logic.
3. Apply "At least" for a Minimum Threshold.
4. Apply "Range of values" for a specific Range filter.

Output:



Result:

Successfully implemented condition-based filtering for precise numeric thresholding.

PROGRAM - 6

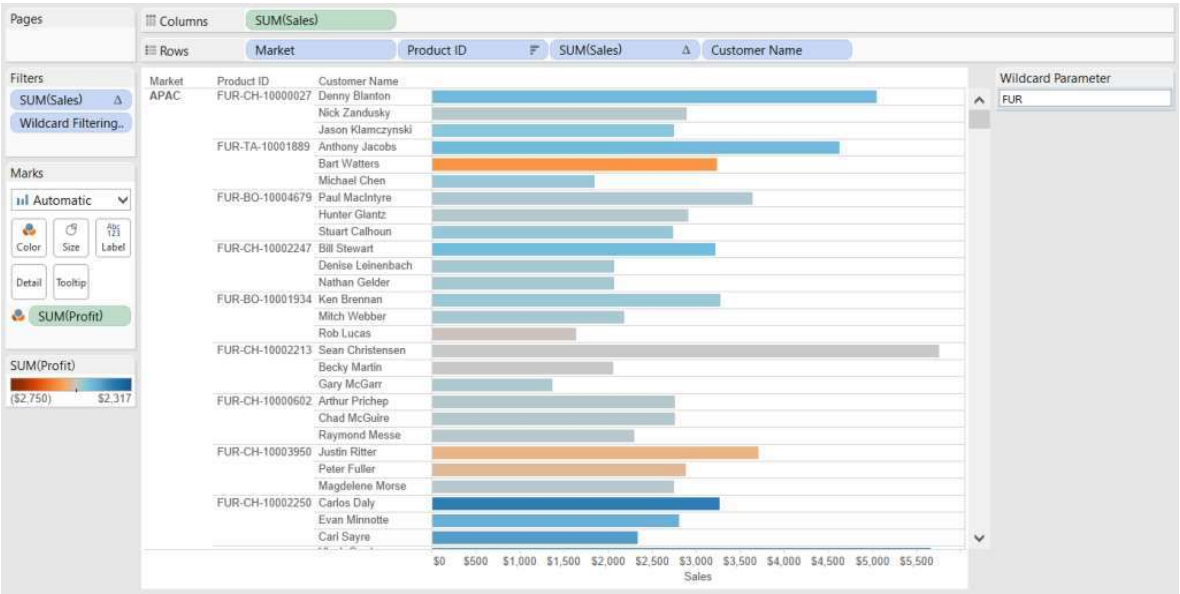
Aim:

To implement and modify quick filters (Single Value, Multiple values, Wildcard Match).

Procedure:

1. Right-click a dimension in the view and select "Show Filter".
2. Click the drop-down arrow on the filter card.
3. Toggle between "Single Value (List)", "Multiple Values (Dropdown)", and "Wildcard Match".

Output:



Result:

Successfully implemented dynamic interactive filters allowing end-user slicing.

PROGRAM - 7

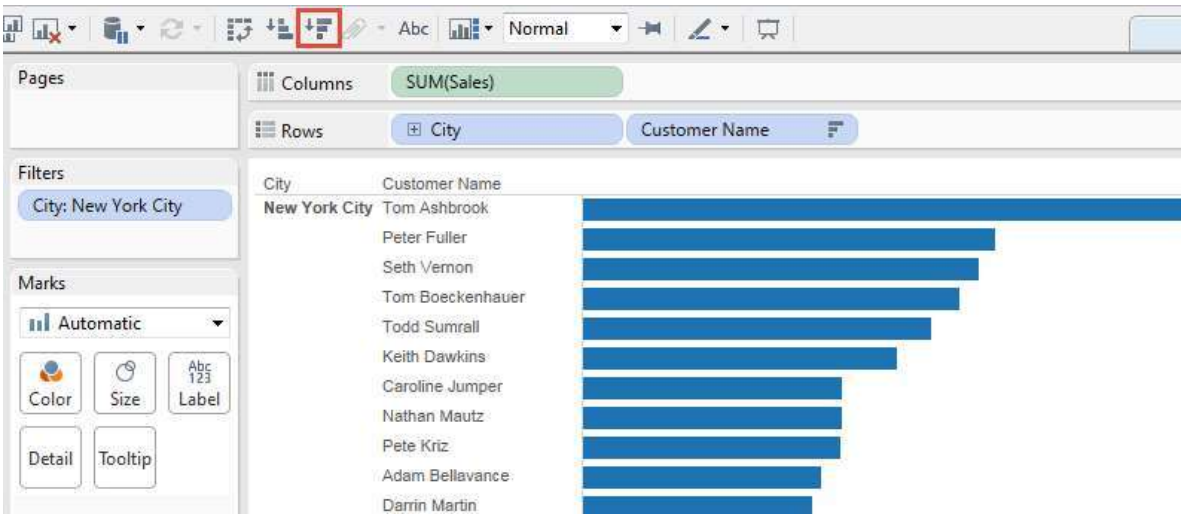
Aim:

To implement a context filter to dictate the order of filtering operations.

Procedure:

1. Add a standard attribute filter to the Filters shelf.
2. Add a second attribute filter.
3. Right-click the primary filter and select "Add to Context" to force it to process first.
4. To remove, right-click and select "Remove from Context".

Output:



Result:

Successfully implemented context filters to limit database querying pipelines.

PROGRAM - 8

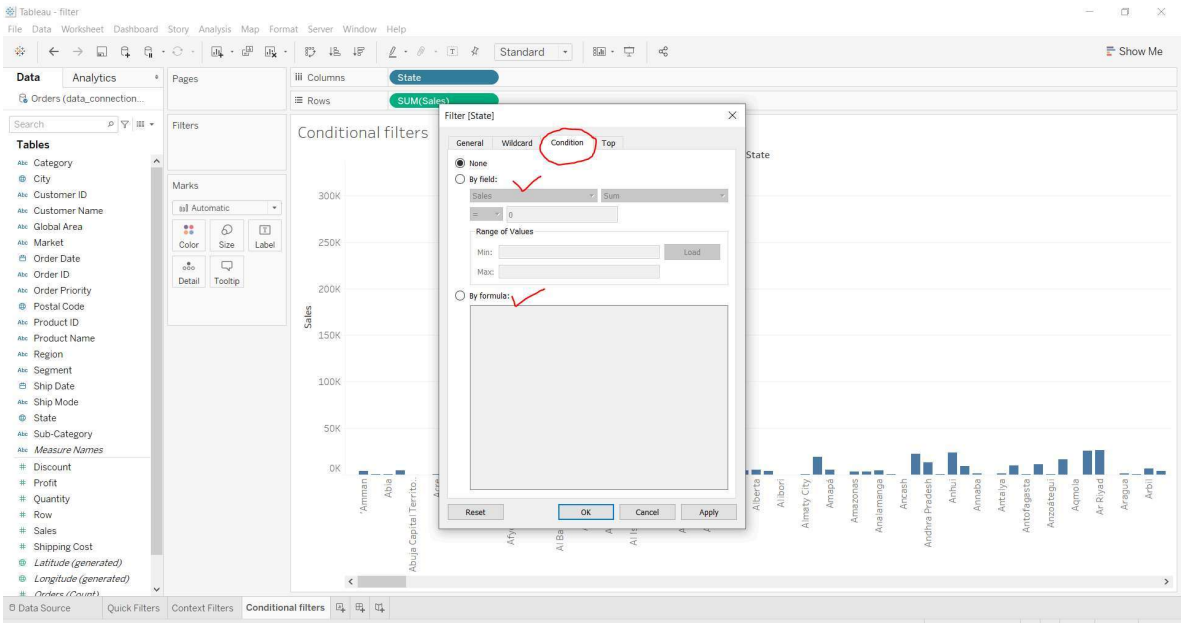
Aim:

To implement conditional filters by field and by mathematical formula.

Procedure:

1. Drag a dimension to the Filters shelf.
2. Switch to the "Condition" tab in the filter dialog.
3. Configure logic By Field (e.g., Sales > 1000).
4. Configure logic By Formula utilizing custom Tableau syntax.

Output:



Result:

Successfully implemented logic-driven conditional filtration on dataset dimensions.

PROGRAM - 9

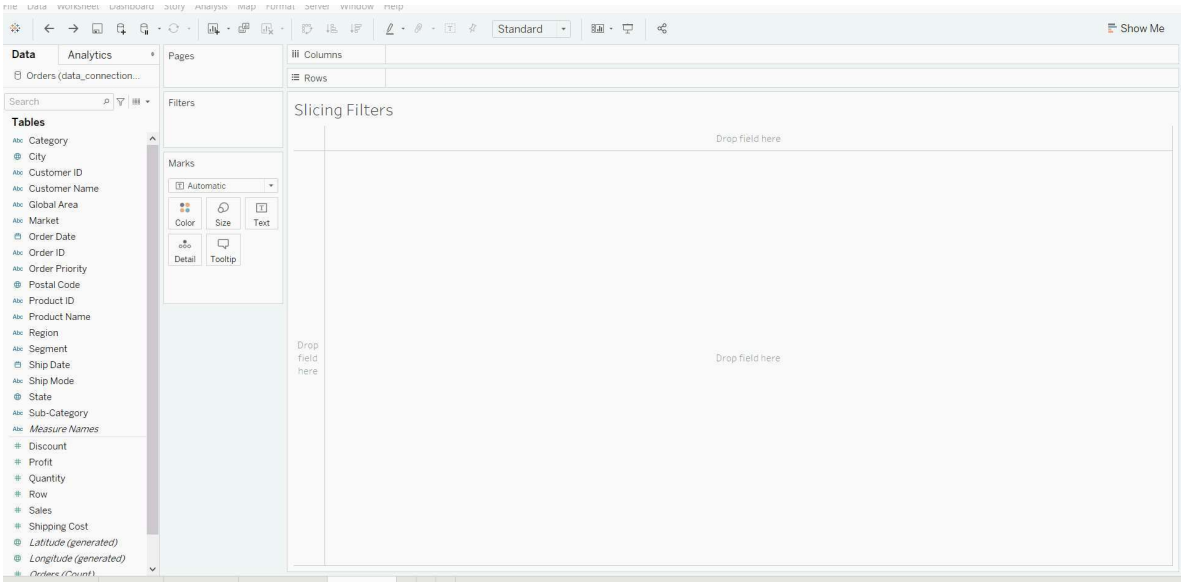
Aim:

To implement slicing filters by dragging to shelves and from attributes.

Procedure:

1. Slice the view by dragging a dimension directly to the Filters shelf.
2. Select specific attributes to include/exclude.
3. Alternatively, right click a mark in the view and select "Keep Only" or "Exclude".

Output:



Result:

Successfully implemented graphical slicing of data across specific dimensions.

PROGRAM - 10

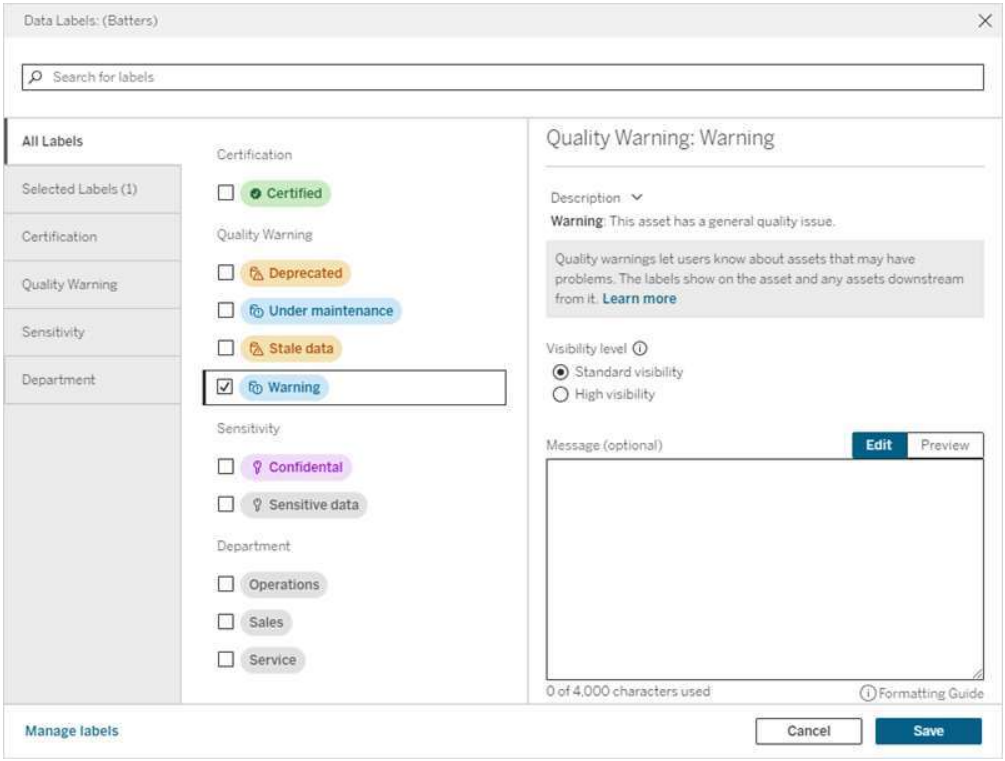
Aim:

To implement data grouping manually and by selecting visual labels.

Procedure:

1. Right-click a dimension in the Data pane and select Create > Group.
2. Manually select items and hit "Group".
3. Alternatively, select multiple data marks (labels) directly in the chart and click the paperclip icon.

Output:



Result:

Successfully implemented category consolidation using ad-hoc group objects.

PROGRAM - 11

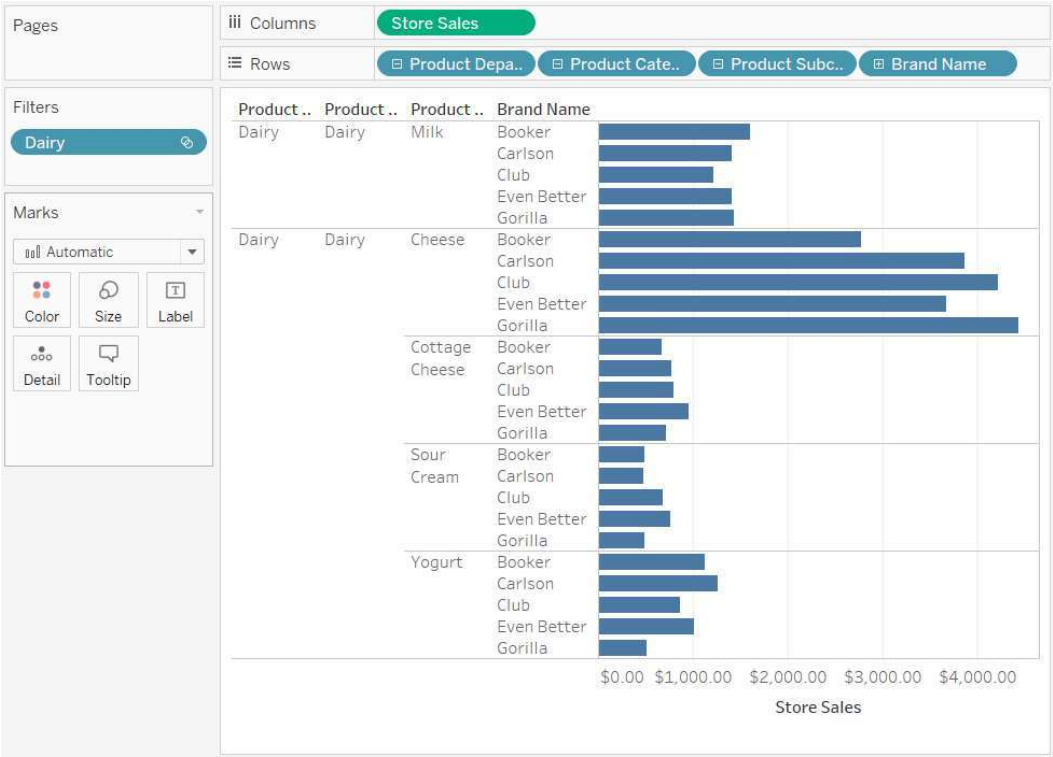
Aim:

To implement and manage data sets dynamically in Tableau.

Procedure:

1. Create a set from an attribute (Create > Set).
2. Create a set directly by selecting visual marks.
3. Drag the Set to color or filter shelves for analysis.
4. Combine two sets or delete unwanted sets.

Output:



Result:

Successfully implemented binary set partitions for advanced segment analysis.

PROGRAM - 12

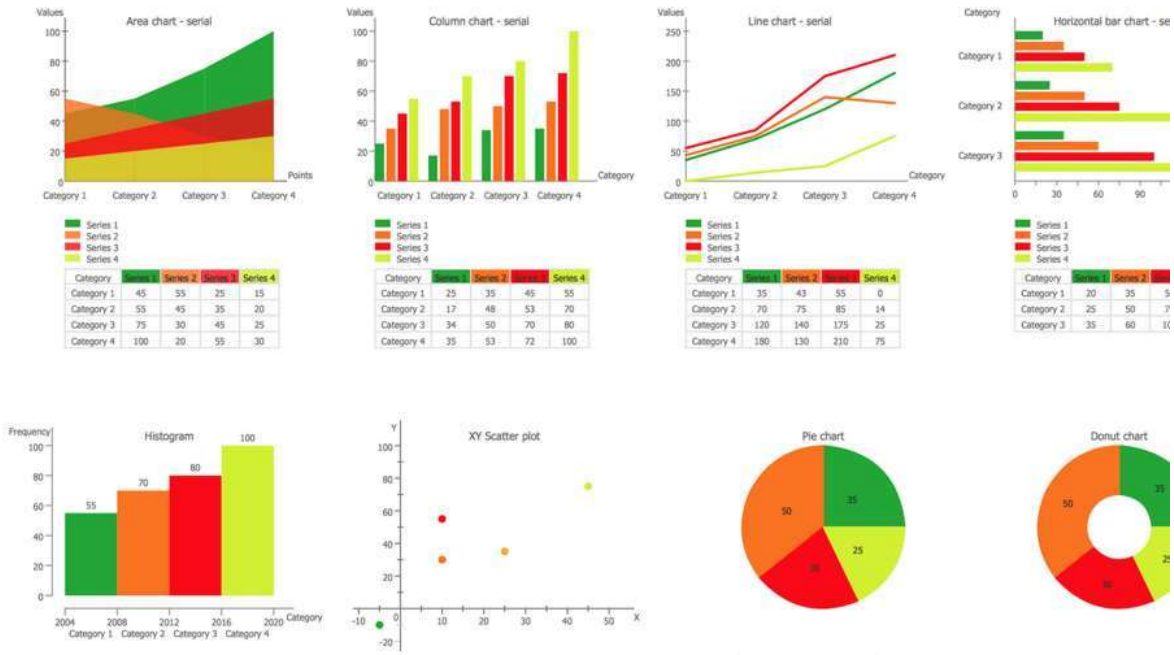
Aim:

To implement various chart types (Bar, Line, Scatter, Map, Pie, Tree, Dual) in Tableau.

Procedure:

1. Drag fields to Rows and Columns shelves manually.
2. Alternatively, use the "Show Me" panel at the top right.
3. Select the required Dimensions and Measures.
4. Click the desired chart type icon to render it instantly.

Output:



Result:

Successfully implemented a complete suite of standard data visualization charts.

PROGRAM - 13

Aim:

To implement data cleaning operations natively within Tableau.

Procedure:

1. Navigate to the Data Source tab.
2. Click "Use Data Interpreter" to clean messy headers.
3. Right-click columns to Split, Rename, or alter Data Types.
4. Use Aliases to rename specific data values.

Output:

Result:

Successfully implemented transformations transforming raw inputs to clean structures.

PROGRAM - 14

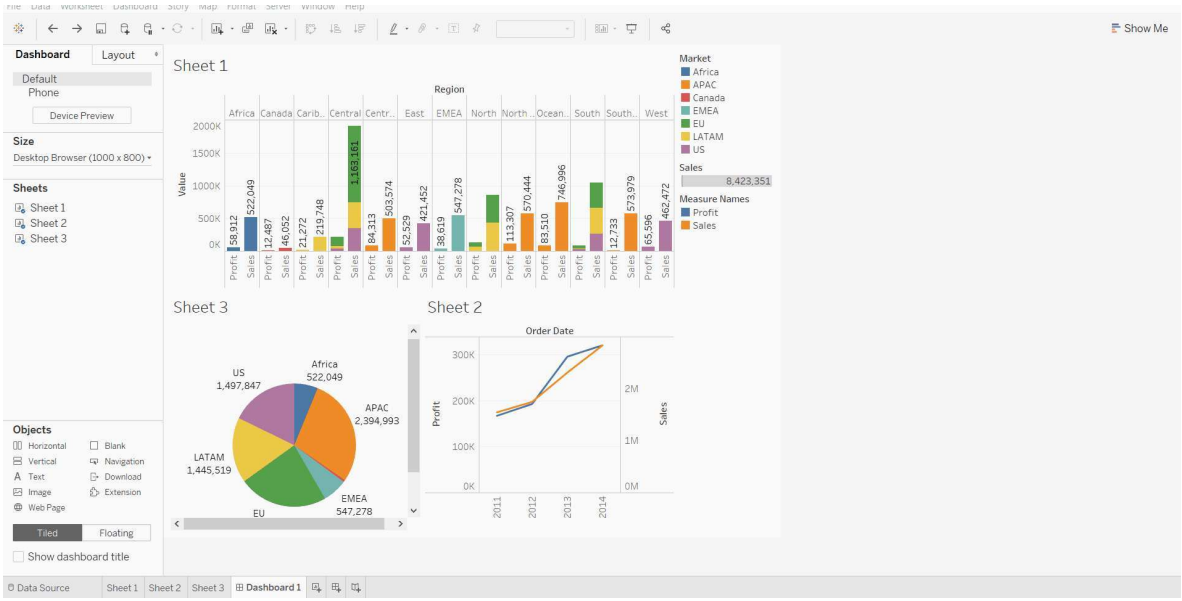
Aim:

To implement, organize, and format an interactive Dashboard.

Procedure:

1. Click the "New Dashboard" icon.
2. Drag created worksheets from the left pane onto the dashboard canvas.
3. Use layout containers (Vertical/Horizontal) to align charts.
4. Enable "Use as Filter" on a chart to make the dashboard interactive.

Output:



Result:

Successfully implemented an aligned, interactive dashboard for visual reporting.

PROGRAM - 15

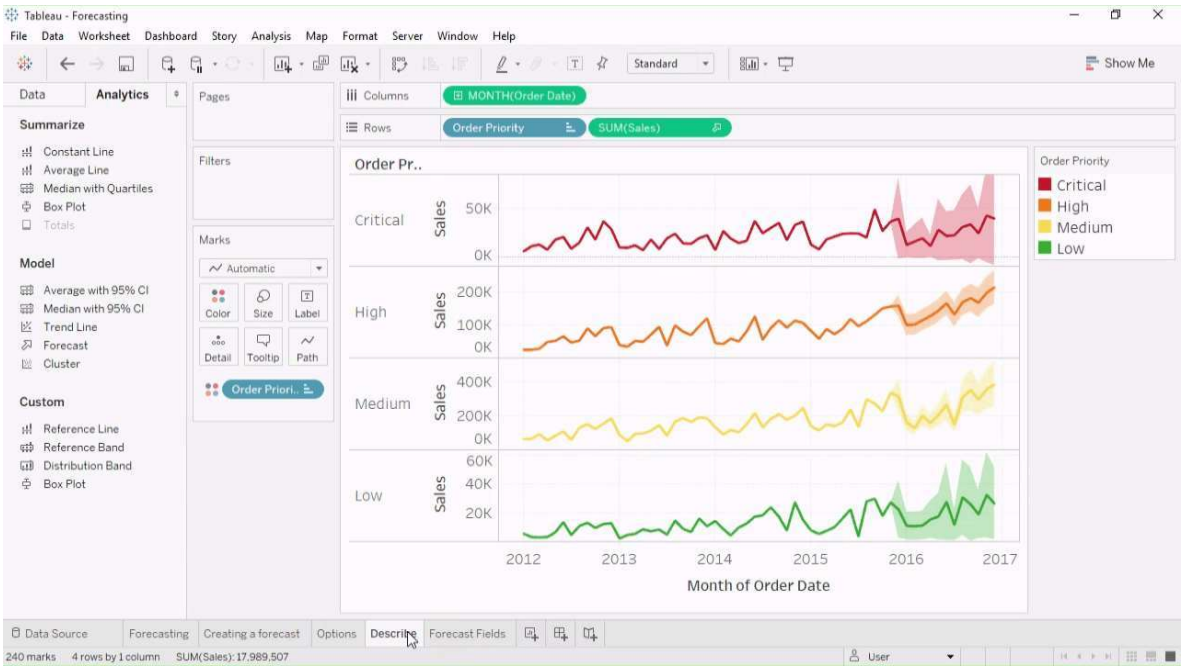
Aim:

To implement, edit, and remove statistical forecasting in Tableau.

Procedure:

1. Create a line chart with a Date field and a Measure.
2. Switch to the Analytics Pane on the left.
3. Drag "Forecast" onto the view.
4. Right-click the forecast line to Edit or Remove.

Output:



Result:

Successfully implemented predictive forecasting models natively on time-series data.

PROGRAM - 16

Aim:

To implement a Histplot using seaborn in Python.

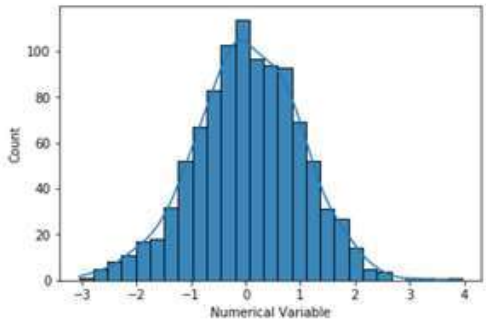
Procedure:

1. Import seaborn and matplotlib libraries.
2. Load a sample dataset (e.g., 'tips').
3. Use `sns.histplot()` to plot the distribution of a variable.
4. Display the plot using `plt.show()`.

Code:

```
import seaborn as sns
import matplotlib.pyplot as plt

def create_histplot():
    # Load dataset
    df = sns.load_dataset("tips")

    # Generate histplot
    sns.histplot(data=df, x="total_bill", bins=20, color='blue')

    plt.title("Histogram of Total Bill")
    plt.xlabel("Total Bill")
    plt.ylabel("Frequency")
    plt.show()

if __name__ == '__main__':
    create_histplot()
```

Output:



Result:

Successfully implemented a histogram plot using Seaborn syntax.

PROGRAM - 17

Aim:

To implement random normal distribution generation and plot a distplot using seaborn.

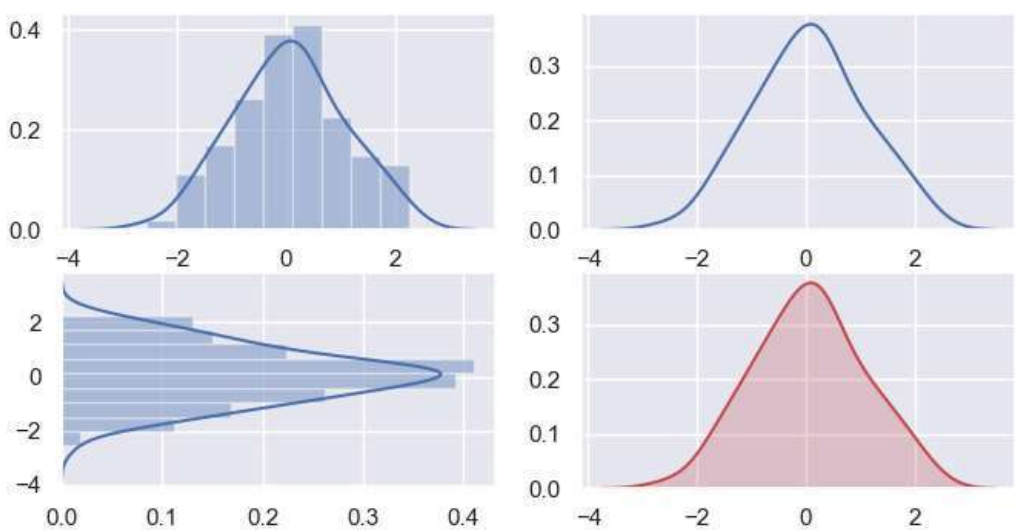
Procedure:

1. Import numpy, seaborn, and matplotlib.
2. Generate random normal data using `np.random.normal()`.
3. Plot the data using Seaborn's density plot functionality.

Code:

```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

def create_normal_distplot():
    # Create random values in normal distribution
    data = np.random.normal(loc=0, scale=1, size=1000)

    # Create distplot (using histplot with KDE for modern seaborn)
    sns.histplot(data, kde=True, color='purple', stat="density")

    plt.title("Normal Distribution Distplot")
    plt.show()

if __name__ == '__main__':
    create_normal_distplot()
```

Output:



Result:

Successfully implemented randomized data distributions and plotted density curves.

PROGRAM - 18

Aim:

To implement a lineplot by loading a dataset using pyplot and seaborn.

Procedure:

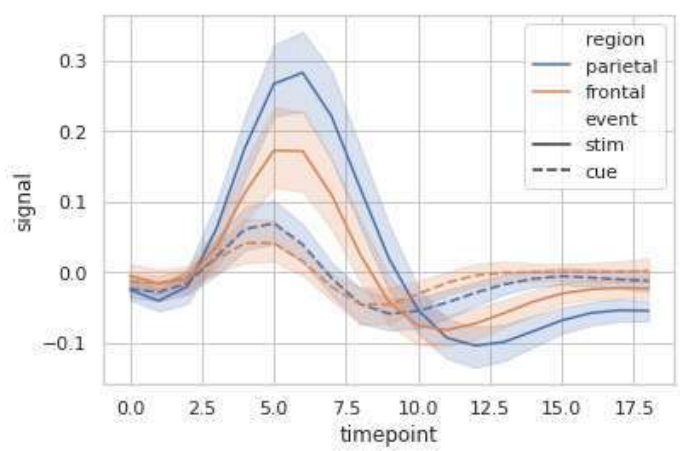
1. Load a pre-configured dataset (e.g., 'flights') via seaborn.
2. Use `sns.lineplot()` assigning x and y axes.
3. Display the line plot with matplotlib.

Code:

```
import seaborn as sns
import matplotlib.pyplot as plt

def create_lineplot():
    # Load dataset
    flights = sns.load_dataset("flights")

    # Create lineplot
    sns.lineplot(data=flights, x="year", y="passengers", hue="month")

    plt.title("Flight Passengers Over Years")
    plt.xlabel("Year")
    plt.ylabel("Number of Passengers")
    plt.show()

if __name__ == '__main__':
    create_lineplot()
```

Output:

```
<matplotlib.axes._subplots.AxesSubplot at 0x7fc9b849fc18>
```



Result:

Successfully implemented a continuous data lineplot using a loaded dataframe.

PROGRAM - 19

Aim:

To implement linear regression visualization in Python.

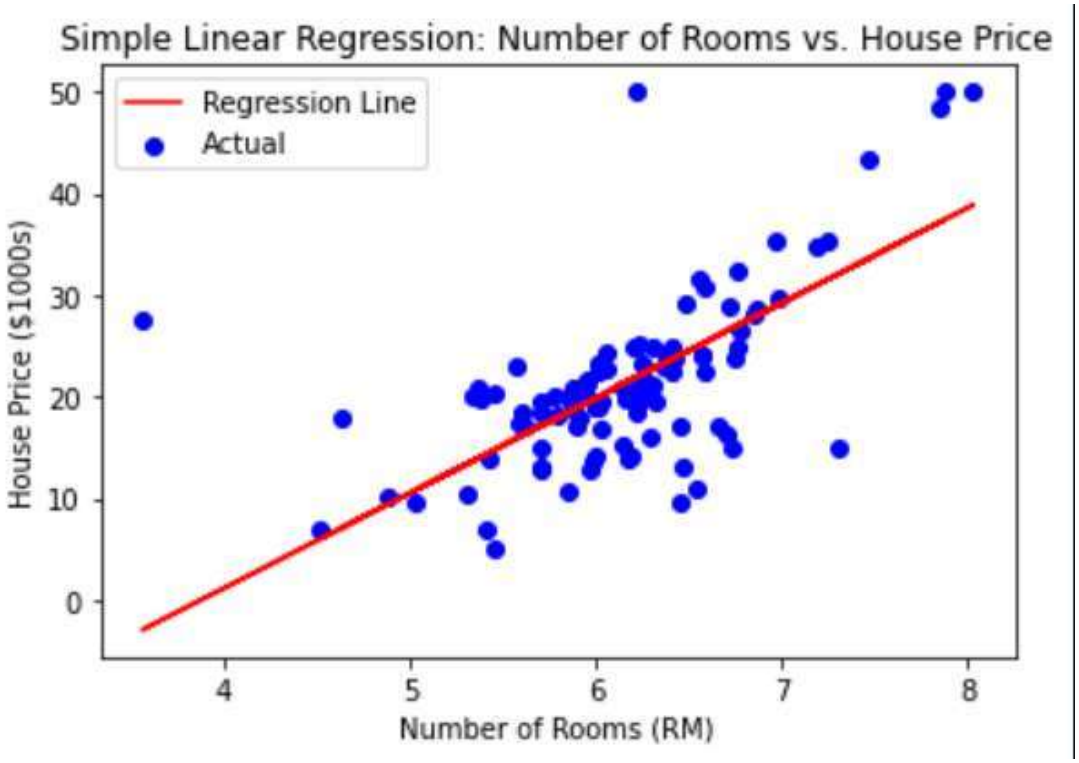
Procedure:

1. Import seaborn and load a dataset containing correlated numeric variables.
2. Use `sns.regplot()` or `sns.lmplot()` to automatically calculate and plot the regression line.
3. Display the figure.

Code:

```
import seaborn as sns
import matplotlib.pyplot as plt

def plot_linear_regression():
    # Load sample dataset
    tips = sns.load_dataset("tips")

    # Plot linear regression line over scatter plot
    sns.regplot(data=tips, x="total_bill", y="tip", scatter_kws=
{'alpha':0.5})

    plt.title("Linear Regression: Total Bill vs Tip")
    plt.show()

if __name__ == '__main__':
    plot_linear_regression()
```

Output:



Result:

Successfully implemented statistical linear regression rendering on a dataset.

PROGRAM - 20

Aim:

To implement a stripplot using pyplot and seaborn.

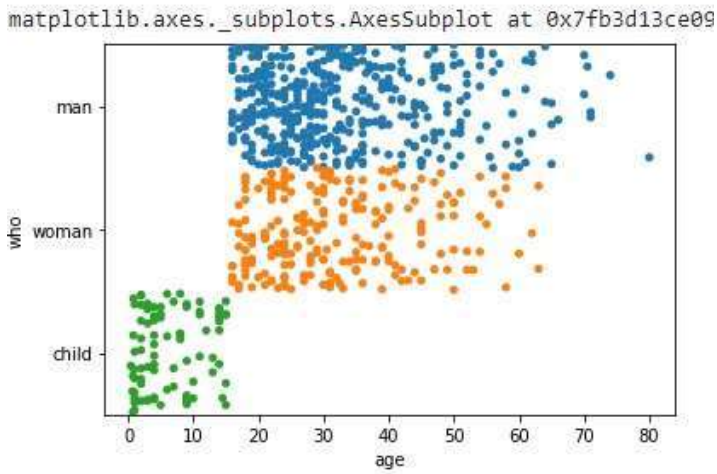
Procedure:

1. Load categorical and numeric data fields.
2. Use `sns.stripplot()` to draw a scatterplot where one variable is categorical.
3. Add jitter to prevent point overlap.

Code:

```
import seaborn as sns
import matplotlib.pyplot as plt

def create_stripplot():
    # Load dataset
    tips = sns.load_dataset("tips")

    # Create stripplot
    sns.stripplot(data=tips, x="day", y="total_bill", jitter=True,
palette="Set2")

    plt.title("Strip Plot of Bills by Day")
    plt.show()

if __name__ == '__main__':
    create_stripplot()
```

Output:



Result:

Successfully implemented a jittered strip plot visualizing categorical dispersion.

PROGRAM - 21

Aim:

To implement a barplot and boxplot in Python.

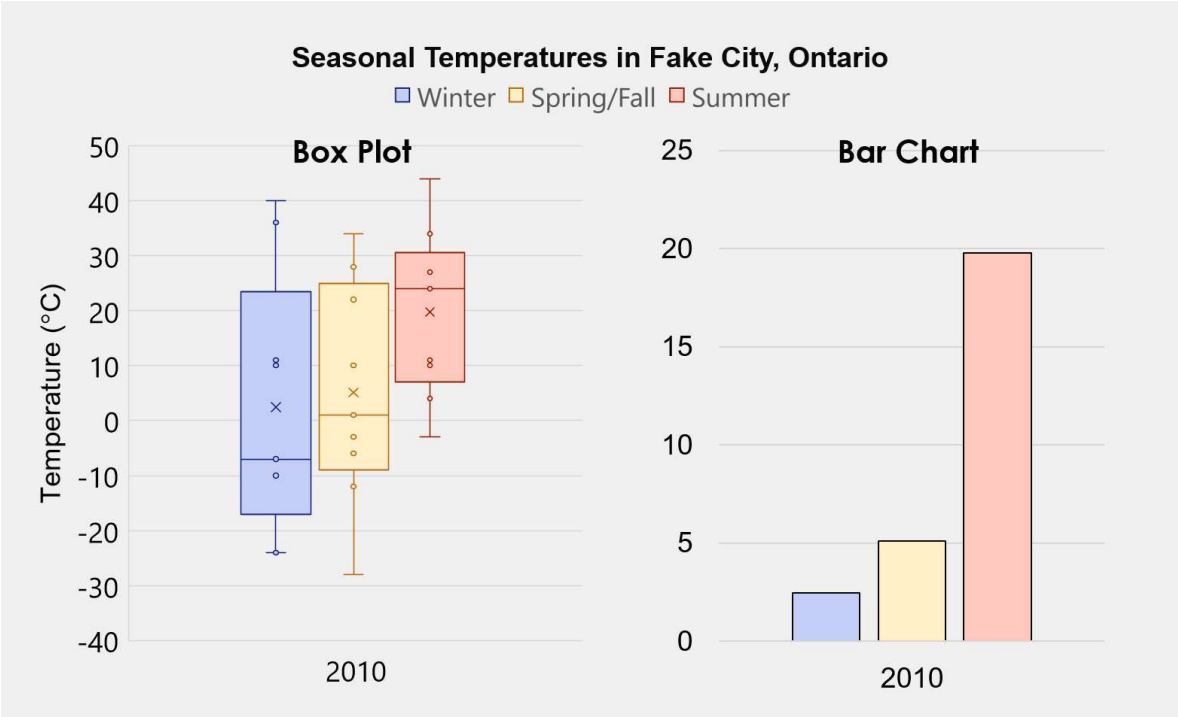
Procedure:

1. Set up a matplotlib figure with subplots for side-by-side rendering.
2. Use `sns.barplot()` on the first axis.
3. Use `sns.boxplot()` on the second axis to show quartile ranges.

Code:

```
import seaborn as sns
import matplotlib.pyplot as plt

def plot_bar_and_box():
    tips = sns.load_dataset("tips")

    # Set up 1x2 figure
    fig, axes = plt.subplots(1, 2, figsize=(10, 5))

    # Barplot
    sns.barplot(data=tips, x="day", y="total_bill", ax=axes[0])
    axes[0].set_title("Barplot")

    # Boxplot
    sns.boxplot(data=tips, x="day", y="total_bill", ax=axes[1])
    axes[1].set_title("Boxplot")

    plt.tight_layout()
    plt.show()

if __name__ == '__main__':
    plot_bar_and_box()
```

Output:



Result:

Successfully implemented aggregation bar plots alongside interquartile box plots.

PROGRAM - 22

Aim:

To implement a palplot using color_palette in seaborn.

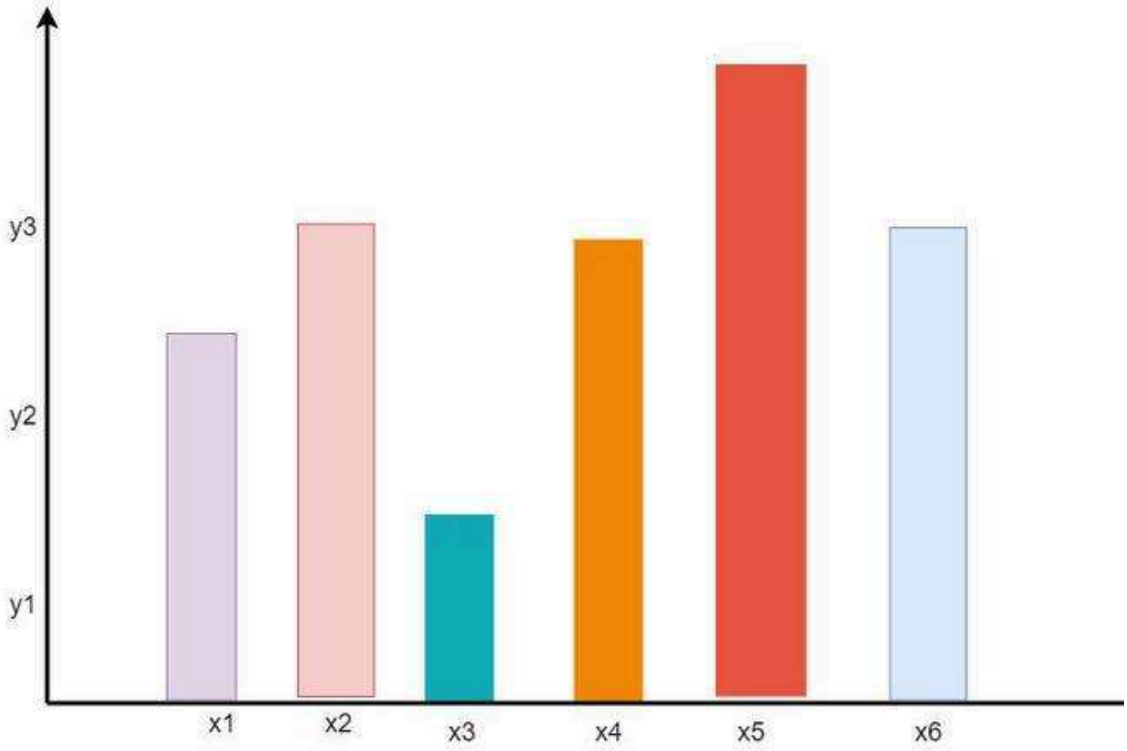
Procedure:

1. Load the seaborn module.
2. Generate a color palette object using `sns.color_palette()`.
3. Render the palette horizontally using `sns.palplot()`.

Code:

```
import seaborn as sns
import matplotlib.pyplot as plt

def show_palette():
    # Define a custom color palette
    custom_palette = sns.color_palette("husl", 8)

    # Display the palette as a plot
    sns.palplot(custom_palette)

    plt.title("Seaborn 'husl' Color Palette", pad=20)
    plt.show()

if __name__ == '__main__':
    show_palette()
```

Output:



Result:

Successfully implemented rendering of hex color swatches using palette plotting.

PROGRAM - 23

Aim:

To implement mean, median, mode, standard deviation, and variance calculations.

Procedure:

1. Import numpy for basic stats and scipy.stats for mode calculation.
2. Create an array of numeric data.
3. Call `mean`, `median`, `mode`, `std`, and `var` functions respectively.

Code:

```
import numpy as np
from scipy import stats

def compute_statistics():
    data = [10, 20, 20, 30, 40, 50, 60]

    print("Dataset:", data)
    print("Mean     :", np.mean(data))
    print("Median   :", np.median(data))

    # Mode returns an object, we extract the scalar value
    mode_result = stats.mode(data, keepdims=True)
    print("Mode     :", mode_result.mode[0])

    print("Std Dev  :", np.round(np.std(data), 2))
    print("Variance :", np.round(np.var(data), 2))

if __name__ == '__main__':
    compute_statistics()
```

Output:

```
Dataset: [10, 20, 20, 30, 40, 50, 60]
Mean     : 32.857142857142854
Median   : 30.0
Mode     : 20
Std Dev  : 16.66
Variance : 277.55
```

Result:

Successfully implemented array-based descriptive statistical calculations.

PROGRAM - 24

Aim:

To implement a correlation matrix using Python.

Procedure:

1. Import pandas and seaborn.
2. Create a DataFrame with multiple numeric columns.
3. Compute correlation using `df.corr()`.
4. Visualize the matrix using `sns.heatmap()`.

Code:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

def create_correlation_matrix():
    # Generate mock dataframe
    data = {
        'Var_A': [1, 2, 3, 4, 5],
        'Var_B': [5, 4, 3, 2, 1],
        'Var_C': [2, 3, 2, 4, 3]
    }
    df = pd.DataFrame(data)

    # Calculate matrix
    corr_matrix = df.corr()
    print("Correlation Matrix:\n", corr_matrix)

    # Plot Heatmap
    sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', vmin=-1,
vmax=1)
    plt.title("Correlation Matrix Heatmap")
    plt.show()

if __name__ == '__main__':
    create_correlation_matrix()
```

Output:

```
Dataframe is :
    x   y   z
0  45  38  10
1  37  31  15
2  42  26  17
3  35  28  21
4  39  33  12
Correlation matrix is :
          x         y         z
x  1.000000  0.518457 -0.701886
y  0.518457  1.000000 -0.860941
z -0.701886 -0.860941  1.000000
```

Result:

Successfully implemented a Pearson correlation matrix and rendered a visual heatmap.

PROGRAM - 25

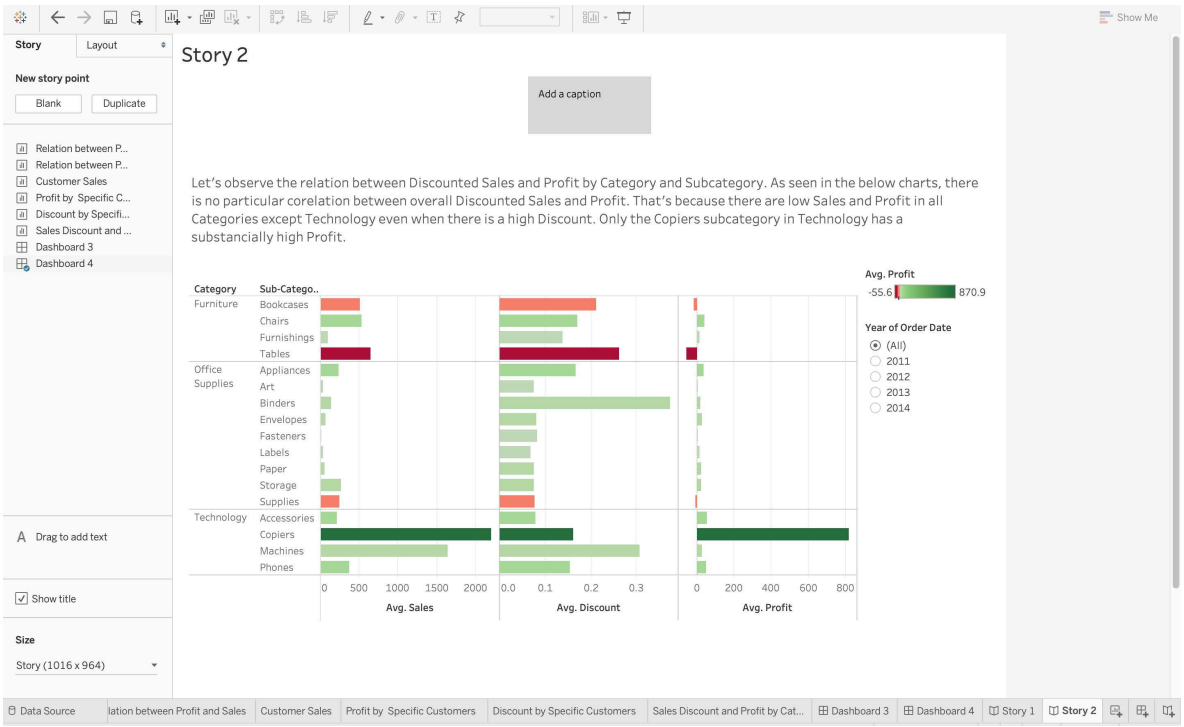
Aim:

To implement a presentation Story in Tableau.

Procedure:

1. Click the "New Story" tab icon at the bottom.
2. Drag worksheets or dashboards to the main view to create Story Points.
3. Add caption text to the top navigator for each point.
4. Click "New Blank Point" to continue the narrative slide deck.

Output:



Result:

Successfully implemented interactive story points for narrative data presentation.